

Laboratorio 2: Processamento de Bases de Dados em Lote

Letícia da Costa Venancio - 250125

```
library(readr)
library(dplyr)
```

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($ARRIVAL_DELAY > 10$)?

É preciso ser calculada uma proporção de todos os voos de uma certa Cia. Aérea em um certo dia, quantos tiveram atraso na chegada maior que 10 minutos. O que é possível fazer calculando uma média de valores 0 e 1 (0 se o voo não atrasou mais que 10 minutos, 1 se atrasou).

Para obter essa média sem a necessidade de ler a base de dados inteira, o cálculo pode ser dividido em:

- n : número de voos daquele grupo com informação válida de atraso (não faltantes)
- n_atraso : número desses voos que tiveram atraso maior que 10 minutos

Como essas variáveis são somáveis, ao processar o arquivo em partes, é possível calcular n e n_atraso de cada parte separadamente, e ao final juntar todos os valores. Assim, o percentual de voos atrasados na chegada é calculado por

$$\text{Perc} = \text{soma}(n_atraso) / \text{soma}(n)$$

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):

- a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;

- b. Remova observações que tenham valores faltantes em campos de interesse;
- c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
- d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe tibble;
- e. A função deve receber apenas dois argumentos:
 - i. input: o conjunto de dados (referente ao lote em questão);
 - ii. pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos){  
  input %>%  
    filter(AIRLINE %in% c('AA','DL','UA','US')) %>%  
    filter(!is.na(ARRIVAL_DELAY)) %>%  
    group_by(YEAR, MONTH, DAY, AIRLINE) %>%  
    summarise(n = n(),  
              n_atraso = sum(ARRIVAL_DELAY > 10),  
              .groups = 'drop')  
}
```

3. Utilize alguma função readr::read_***_chunked para importar o arquivo flights.csv.zip.

- a. Configure o tamanho do lote (chunk) para 100 mil registros;
- b. Configure a função de callback para instanciar DataFrames aplicando a função getStats criada acima;

c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
mycols <- cols_only(YEAR = 'i',
                   MONTH = 'i',
                   DAY = 'i',
                   AIRLINE = 'c',
                   ARRIVAL_DELAY = 'i')

stats_lotes <- read_csv_chunked('flights.csv.zip',
                               callback = DataFrameCallback$new(getStats),
                               chunk_size = 1e5,
                               col_types = mycols)
```

4. Crie uma função chamada `computeStats` que:

a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);

b. Retorne as informações em um tibble contendo apenas as seguintes colunas:

i. Cia: sigla da companhia aérea;

ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);

iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1]

```
computeStats <- function(stats){
  stats %>%
    group_by(YEAR, MONTH, DAY, AIRLINE) %>%
    summarise(n = sum(n),
              n_atraso = sum(n_atraso),
              .groups = 'drop') %>%
    mutate(Data = as.Date(paste(YEAR, MONTH, DAY, sep = '-')),
           Perc = n_atraso / n,
           Cia = AIRLINE) %>%
    select(Cia, Data, Perc)
```

```
}

resultado <- stats_lotes %>% computeStats()
resultado
```

```
# A tibble: 1,276 x 3
  Cia    Data      Perc
  <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 DL    2015-01-01 0.0847
3 UA    2015-01-01 0.289
4 US    2015-01-01 0.196
5 AA    2015-01-02 0.430
6 DL    2015-01-02 0.180
7 UA    2015-01-02 0.349
8 US    2015-01-02 0.195
9 AA    2015-01-03 0.538
10 DL   2015-01-03 0.347
# i 1,266 more rows
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

a. Instale e carregue os pacotes ggcal e ggplot2.

```
library(ggplot2)
library(lubridate)
```

Como o pacote ggcal não está mais no CRAN, usei apenas o pacote ggplot2 e o pacote lubridate para algumas funções.

b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.

```
pal <- scale_fill_gradient(low = '#4575b4', high = '#d73027', na.value = 'white')
```

c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:

i. Criar um subconjunto de stats de forma a conter informações de atraso e data apenas da Cia. Aérea dada por cia.

ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, x representa as datas de interesse e y, os percentuais de atraso para as datas descritas em x.

iii. Retornar para o usuário a base do calendário criada acima.

```
baseCalendario <- function(stats, cia){
  sub <- stats %>%
    filter(Cia == cia) %>%
    mutate(dow = wday(Data, label = TRUE, week_start = 7),
           month_lab = month(Data, label = TRUE, abbr = FALSE),
           first_dow = wday(floor_date(Data, "month"), week_start = 7),
           week_of_month = ceiling((day(Data) + first_dow - 1) / 7))

  ggplot(sub, aes(x = dow, y = -week_of_month, fill = Perc)) +
    geom_tile(color = "white") +
    facet_wrap(~ month_lab, scales = "free", ncol = 3) +
    theme_minimal() +
    theme(axis.title = element_blank(),
          axis.text.y = element_blank(),
          panel.grid = element_blank())
}
```

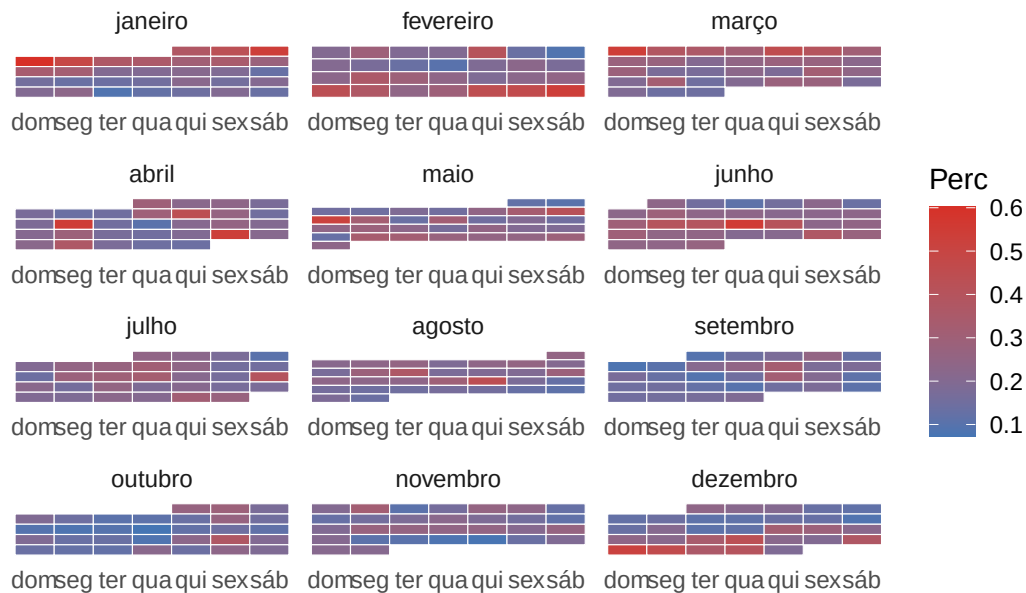
d. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.

```
cAA <- baseCalendario(resultado, 'AA')
cDL <- baseCalendario(resultado, 'DL')
cUA <- baseCalendario(resultado, 'UA')
cUS <- baseCalendario(resultado, 'US')
```

e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Titulo")`.

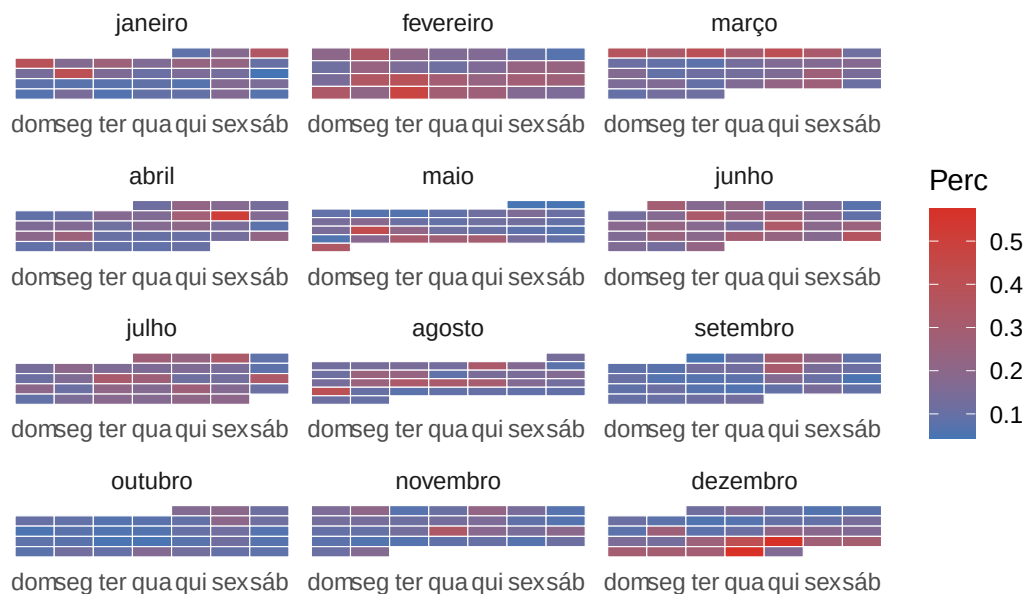
```
cAA + pal + ggtitle("American Airlines - % de voos atrasados (>10 min)")
```

American Airlines – % de voos atrasados (>10 min)



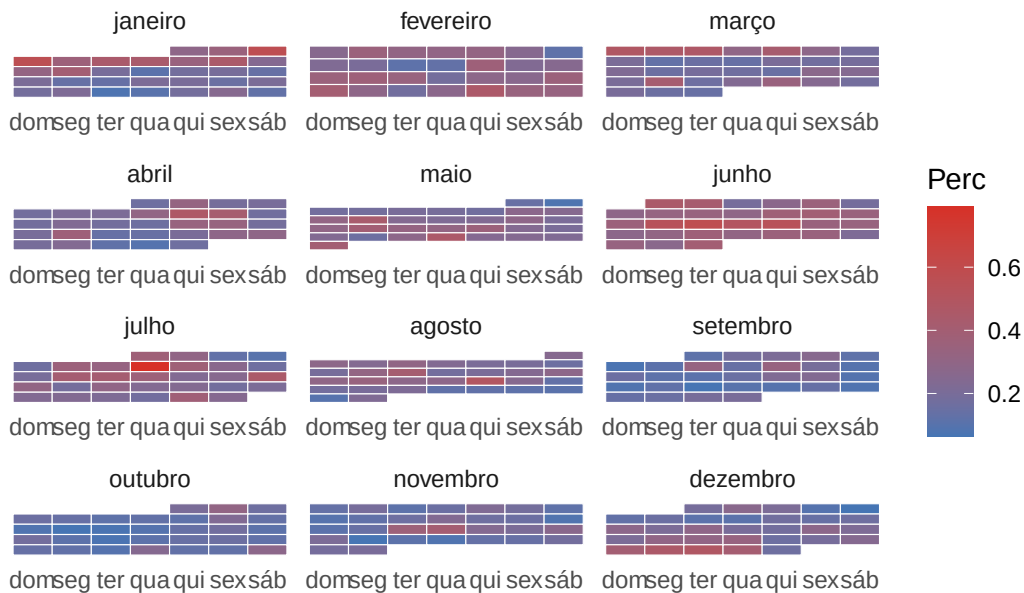
```
cDL + pal + ggtitle("Delta Air Lines - % de voos atrasados (>10 min)")
```

Delta Air Lines – % de voos atrasados (>10 min)



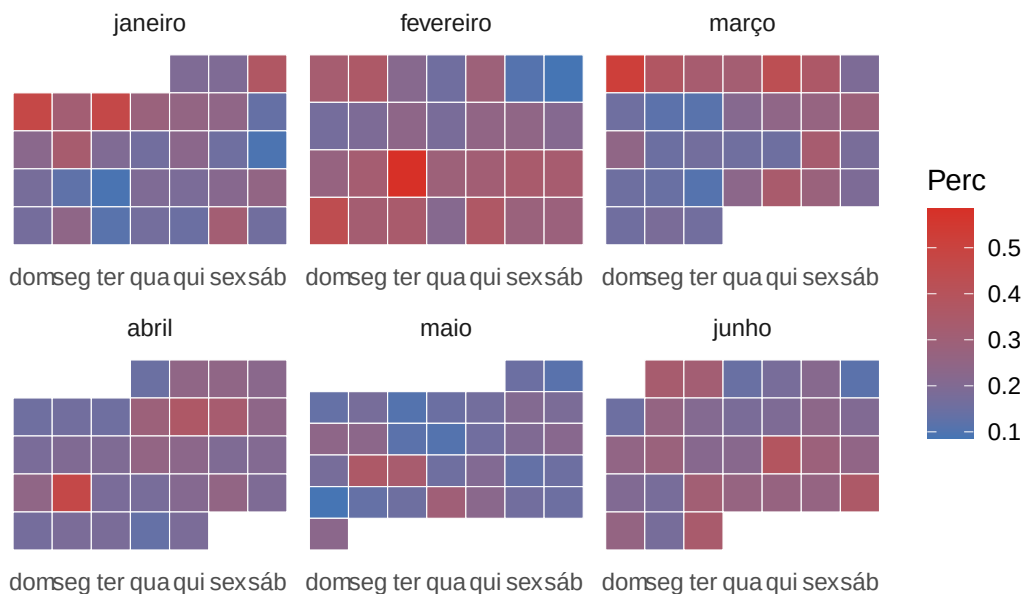
```
cUA + pal + ggtitle("United Airlines - % de voos atrasados (>10 min)")
```

United Airlines – % de voos atrasados (>10 min)



```
cUS + pal + ggtitle("US Airways - % de voos atrasados (>10 min)")
```

US Airways – % de voos atrasados (>10 min)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 23:47:46 -03"
```